25 FEB 2026

AADK TASK 2

Design a Kotlin Class & Data Model for an App Feature

MindMatrix

Chosen Feature: Task Tracker App

Feature Description

I have chosen a Task Tracker feature.
This feature allows users (students or working professionals) to create tasks, mark them as completed, and organize their daily activities. It helps users stay productive and manage time effectively. This feature is important because it improves task organization and keeps track of pending and completed work.

Modeling the Data

Data Class

```
data class Task(
    val id: Int,
    var title: String,
    var description: String,
    var isCompleted: Boolean
)
```

Interface for Behavior

```
interface TaskManager {
    fun addTask(task: Task)
    fun removeTask(task: Task)
    fun toggleTaskCompletion(task: Task)
}
```

Class Implementing the Interface

```
class TaskRepository : TaskManager {

    private val taskList = mutableListOf<Task>()

    override fun addTask(task: Task) {
        taskList.add(task)
    }

    override fun removeTask(task: Task) {
        taskList.remove(task)
    }

    override fun toggleTaskCompletion(task: Task) {
        task.isCompleted = !task.isCompleted
    }

    fun getAllTasks(): List<Task> {
        return taskList
    }
}
```

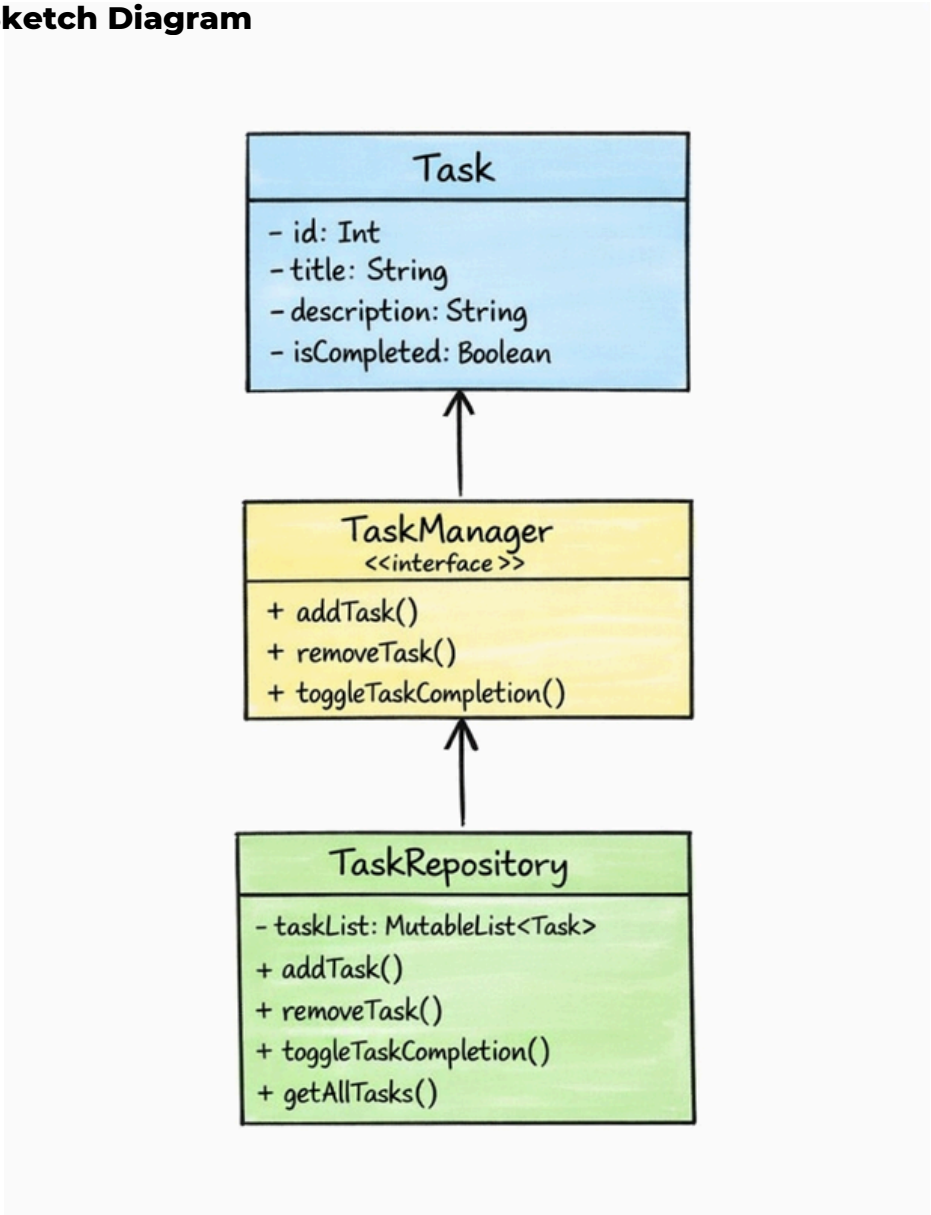
Explanation

I chose:

- Data class for Task because it mainly holds data (title, description, completion status). Data classes automatically provide toString(), equals(), and copy() functions, which is very useful.
- Interface for TaskManager because it defines behavior (what actions can be done with tasks) without implementing them.
- Regular class (TaskRepository) to implement the actual logic and manage the task list.

This separation makes the code clean, reusable, and follows object-oriented principles.

UML / Sketch Diagram



Kotlin Features Used:

- Primary constructor in Task
- Mutable properties (var isCompleted)
- Interface
- Class implementation
- Encapsulation (private taskList)

Mapping to Compose UI

In Jetpack Compose, I can use a LazyColumn to display a list of tasks:

- Each item in LazyColumn will display title and description.
- A Checkbox or IconButton can be used to toggle isCompleted.
- When toggleTaskCompletion() is called, the state will change.
- If the task list is stored in a ViewModel using mutableStateListOf, Compose will automatically recompose the UI when the task state changes.

This makes UI dynamic and reactive.

Reflection

I found it slightly challenging to model behavior instead of just defining data. Thinking about what actions the user can perform required more planning. However, using object-oriented concepts like classes and interfaces makes the app structure clean and maintainable. It separates UI from business logic, which is very helpful for building scalable Compose apps.